Q1. ax, Autograd and Neural Networks (40 points + 14 Bonus points)

(a) Univariate Function Gradient (5 points)

In [2]:
```
import jax.numpy as jnp
from jax import grad, jit, vmap
from jax import random
import numpy as np

# Q1a.  Example for taking the derivative of a simple function

def fun(x):
    return jnp.cos(x**3 - 2*x**2 + 5)

def manual_grad(x):
    # Implement the derivative w.r.t. x
    # ==================YOUR CODE HERE =================
    gradient=-(3*x**2 - 4*x)*jnp.sin(x**3 - 2*x**2 + 5)
    # ==================================================
    return gradient

grad_fun = grad(fun)

x = 3.5

print("Gradient:", grad_fun(x),  manual_grad(x))
```

```
Gradient: 22.353619 22.353619
```

(b) Multivariate Function Gradient (5 points)

In [6]:
```
# Now let's do it for a multivariate case

#Q1b.  Please complete the manual grad.


def cubic_l2_norm(x):
    return jnp.linalg.norm(x)**3

# Now complete the following line
grad_fun = grad(cubic_l2_norm)


x = jnp.array([5.0,0.3,2])

def manual_grad(x):
    # Manually implement the gradient w.r.t. x
    # ==================YOUR CODE HERE =================
```

```
    gradient = 3 * jnp.linalg.norm(x) * x
    # ==================================================
    return gradient

print("Gradient:", grad_fun(x),  manual_grad(x))
```

```
Gradient: [80.902725  4.854164 32.36109 ] [80.90272   4.854163 32.361088]
```

Implement a function that computes the Hessian matrix for the given function (2 Bonus points)

In [7]:
```
# Code for computing Hessian matrix

from jax import jacfwd

def hessian(fun, x):
    return jacfwd(grad(fun))(x)
```

(c) Soft-Max Regression and Neural Networks (15 points)

In [8]:
```
# Q1c. Neural Network from Scratch
# Example implementation of simple soft-max regression with square loss
#[Nothing to implement in this block]

# define soft argmax
def soft_argmax(x):
    y = jnp.exp(x)
    return y/jnp.sum(y)

# Specify the how to caculate the

def predict(params, inputs):
    W = params[0]
    b = params[1]
    scores = jnp.dot( W, inputs) + b
    outputs = soft_argmax(scores)
    return outputs

def loss_fun(params, inputs, targets):
    preds = predict(params, inputs)
    # You can use cross-entropy loss instead
    return jnp.sum((preds - targets)**2)

grad_fun = grad(loss_fun)  # gradient evaluation function

# Lets' cook up some input for this
W = np.random.randn(3,4)
b = np.random.randn(3,1)
inputs = np.random.randn(4,1)
targets = np.array([1,0,0])
params=[W,b]
```

In [9]:
```
# Example usage of above functions
# [Nothing to implement in this block]

# How to Generate a prediction for this inputs
print(predict(params, inputs))

# How to Compute the gradient from this imput
print(grad_fun(params, inputs,targets))

# How to implement an SGD udpate
# Take a minibatch of data X,y
def sgd_update(params,X,y):
    lr = 0.01
    gradient = grad_fun(params,X,y)
    for param, g in zip(params,gradient):
        param -= lr*g
```

```
[[0.04176054]
 [0.8943134 ]
 [0.063926  ]]
[Array([[ 0.06380932, -0.0391952 ,  0.31464398,  0.10801183],
       [-0.15865272,  0.09745324, -0.7823171 , -0.26855594],
       [ 0.09484337, -0.05825803,  0.467673  ,  0.16054405]],      dtype=float32), A
rray([[-0.19139682],
       [ 0.47588074],
       [-0.28448385]], dtype=float32)]
```

Now you are expected to implement a two-layer neural network from scratch by modifying the above code for soft-max regression.

Very Important the only function you need to modify is `predict_nn`

In [10]:
```
# Q1c.  continues

#  You may use jnp.tanh to implement a hyperbolic tangent activation fuonction.
#

def predict_nn(params, inputs):

    W1 = params[0]
    b1 = params[1]
    W2 = params[2]
    b2 = params[3]

    # ==================YOUR CODE HERE =================

    # implement the score function of a two layer neural networks.

    # ==================================================
    scores = jnp.dot(W2, jnp.tanh(jnp.dot(W1, inputs) + b1)) + b2
    outputs = soft_argmax(scores)
    return outputs


def loss_fun_nn(params, inputs, targets):
```

```
    preds = predict_nn(params, inputs)
    # You can use logistic loss instead
    return jnp.sum((preds - targets)**2)

grad_fun_nn = grad(loss_fun_nn)  # gradient evaluation function
```

(d) Neural Network Parameters and Gradient (15 points)

Now let's say the neural network is supposed to classify

In [15]:
```
# Q1d Lets' cook up some input:

inputs = np.random.randn(64,1)
targets = np.array([1,0,0])

# What could be a valid shape of the following parameters?  Replace the question ma

# ==================YOUR CODE HERE =================

W1 = np.random.randn(16,64)
b1 = np.random.randn(16,1)
W2 = np.random.randn(3,16)
b2 = np.random.randn(3,1)

 # ==================================================

# W1 = np.random.randn(6,2)
# b1 = np.random.randn(6,1)
# W2 = np.random.randn(3,6)
# b2 = np.random.randn(3,1)
params=[W1,b1,W2,b2]
```

In [16]:
```
# Generate a prediction for this inputs
predict_nn(params, inputs)

# Compute the gradient from this imput
grad_fun_nn(params, inputs,targets)
```

Out[16]:
```
[Array([[ 3.9520419e-05,  2.9003813e-05, -6.6996887e-05, ...,
          1.7084896e-05,  3.1353411e-05, -7.7006996e-05],
        [ 6.3852901e-09,  4.6861284e-09, -1.0824646e-08, ...,
          2.7603961e-09,  5.0657514e-09, -1.2441974e-08],
        [ 4.4763241e-05,  3.2851491e-05, -7.5884760e-05, ...,
          1.9351397e-05,  3.5512785e-05, -8.7222827e-05],
        ...,
        [-0.0000000e+00, -0.0000000e+00,  0.0000000e+00, ...,
         -0.0000000e+00, -0.0000000e+00,  0.0000000e+00],
        [ 5.0595496e-05,  3.7131751e-05, -8.5771877e-05, ...,
          2.1872713e-05,  4.0139788e-05, -9.8587196e-05],
        [-1.0198230e-05, -7.4844233e-06,  1.7288521e-05, ...,
         -4.4087510e-06, -8.0907357e-06,  1.9871626e-05]], dtype=float32),
 Array([[-5.2854419e-05],
        [-8.5396561e-09],
        [-5.9866143e-05],
        [ 1.1622906e-06],
        [-1.1434077e-08],
        [-3.6265155e-05],
        [ 0.0000000e+00],
        [ 9.2449477e-03],
        [ 0.0000000e+00],
        [-3.8844921e-02],
        [-2.5581219e-05],
        [ 6.3940212e-07],
        [-4.4703484e-08],
        [ 0.0000000e+00],
        [-6.7666173e-05],
        [ 1.3639063e-05]], dtype=float32),
 Array([[-0.03504986,  0.03506137,  0.03504796, -0.03506092,  0.03506136,
          0.03504662, -0.03506138,  0.0212205 ,  0.03506138, -0.02515433,
          0.03499717,  0.03506117, -0.03506133, -0.03506138, -0.03503529,
          0.03499429],
        [ 0.01267165, -0.01267581, -0.01267097,  0.01267565, -0.01267581,
         -0.01267048,  0.01267582, -0.00767189, -0.01267582,  0.0090941 ,
         -0.0126526 , -0.01267574,  0.0126758 ,  0.01267582,  0.01266639,
         -0.01265156],
        [ 0.02237818, -0.02238553, -0.02237697,  0.02238524, -0.02238552,
         -0.02237611,  0.02238554, -0.01354859, -0.02238554,  0.01606021,
         -0.02234454, -0.0223854 ,  0.02238551,  0.02238554,  0.02236888,
         -0.0223427 ]], dtype=float32),
 Array([[ 0.03506138],
        [-0.01267582],
        [-0.02238554]], dtype=float32)]
```

In [ ]:

In [ ]:

In [1]:
```python
import jax.numpy as jnp
from jax import grad, jit, vmap
from jax import random
```

In [2]:
```python
# A helper function to randomly initialize weights and biases
# for a dense neural network layer
```

```
def random_layer_params(m, n, key, scale=1e-2):
  w_key, b_key = random.split(key)
  return scale * random.normal(w_key, (n, m)), scale * random.normal(b_key, (n,))

# Initialize all layers for a fully-connected neural network with sizes "sizes"
def init_network_params(sizes, key):
  keys = random.split(key, len(sizes))
  return [random_layer_params(m, n, k) for m, n, k in zip(sizes[:-1], sizes[1:], ke

layer_sizes = [784, 512, 512, 10]
step_size = 0.01
num_epochs = 10
batch_size = 128
n_targets = 10
params = init_network_params(layer_sizes, random.PRNGKey(0))
```

In [3]:
```
from jax.scipy.special import logsumexp

def relu(x):
    return jnp.maximum(0, x)

def predict(params, image):
  # per-example predictions
  activations = image
  for w, b in params[:-1]:
    outputs = jnp.dot(w, activations) + b
    activations = relu(outputs)

  final_w, final_b = params[-1]
  logits = jnp.dot(final_w, activations) + final_b
  return logits - logsumexp(logits)
```

In [4]:
```
# This works on single examples
random_flattened_image = random.normal(random.PRNGKey(1), (28 * 28,))
preds = predict(params, random_flattened_image)
print(preds.shape)
```

```
(10,)
```

In [5]:
```
# Doesn't work with a batch
random_flattened_images = random.normal(random.PRNGKey(1), (10, 28 * 28))
try:
  preds = predict(params, random_flattened_images)
except TypeError:
  print('Invalid shapes!')
```

```
Invalid shapes!
```

In [6]:
```
# Let's upgrade it to handle batches using `vmap`

# Make a batched version of the `predict` function
batched_predict = vmap(predict, in_axes=(None, 0))

# `batched_predict` has the same call signature as `predict`
batched_preds = batched_predict(params, random_flattened_images)
print(batched_preds.shape)
```

```
(10, 10)
```

In [7]:
```python
def one_hot(x, k, dtype=jnp.float32):
  """Create a one-hot encoding of x of size k."""
  return jnp.array(x[:, None] == jnp.arange(k), dtype)

def accuracy(params, images, targets):
  target_class = jnp.argmax(targets, axis=1)
  predicted_class = jnp.argmax(batched_predict(params, images), axis=1)
  return jnp.mean(predicted_class == target_class)

def loss(params, images, targets):
  preds = batched_predict(params, images)
  return -jnp.mean(preds * targets)

@jit
def update(params, x, y):
  grads = grad(loss)(params, x, y)
  return [(w - step_size * dw, b - step_size * db)
          for (w, b), (dw, db) in zip(params, grads)]
```

In [8]:
```python
# import os
# import gzip
# import numpy as np
# import urllib.request

# # URLs for the MNIST dataset
# URLS = {
#     'train_images': 'http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz'
#     'train_labels': 'http://yann.lecun.com/exdb/mnist/train-labels-idx1-ubyte.gz'
#     'test_images': 'http://yann.lecun.com/exdb/mnist/t10k-images-idx3-ubyte.gz',
#     'test_labels': 'http://yann.lecun.com/exdb/mnist/t10k-labels-idx1-ubyte.gz',
# }

# def download_and_extract_mnist(url, dest):
#     """Download and extract MNIST files."""
#     if not os.path.exists(dest):
#         urllib.request.urlretrieve(url, dest)
#     with gzip.open(dest, 'rb') as f:
#         return np.frombuffer(f.read(), np.uint8, offset=16).reshape(-1, 784)

# def download_and_extract_labels(url, dest):
#     """Download and extract MNIST label files."""
#     if not os.path.exists(dest):
#         urllib.request.urlretrieve(url, dest)
#     with gzip.open(dest, 'rb') as f:
#         return np.frombuffer(f.read(), np.uint8, offset=8)

# # Download and preprocess data
# train_images = download_and_extract_mnist(URLS['train_images'], '/tmp/train-image
# train_labels = download_and_extract_labels(URLS['train_labels'], '/tmp/train-labe
# test_images = download_and_extract_mnist(URLS['test_images'], '/tmp/t10k-images-i
# test_labels = download_and_extract_labels(URLS['test_labels'], '/tmp/t10k-labels-

import os
import gzip
```

```python
import shutil
import numpy as np
import urllib.request
from urllib.error import URLError
from zipfile import ZipFile
from io import BytesIO

# URL for the MNIST dataset ZIP file
MNIST_ZIP_URL = 'https://data.deepai.org/mnist.zip'

def download_and_extract_mnist_zip(url, dest_folder):
    """Download and extract the MNIST ZIP file."""
    if not os.path.exists(dest_folder):
        os.makedirs(dest_folder)
    zip_path = os.path.join(dest_folder, 'mnist.zip')
    try:
        urllib.request.urlretrieve(url, zip_path)
    except URLError as e:
        print(f"Failed to download the dataset. Error: {e}")
        return False
    with ZipFile(zip_path, 'r') as zip_ref:
        zip_ref.extractall(dest_folder)
    os.remove(zip_path)
    return True

def extract_gz_file(gz_path, dest_path):
    """Extract a .gz file."""
    with gzip.open(gz_path, 'rb') as f_in:
        with open(dest_path, 'wb') as f_out:
            shutil.copyfileobj(f_in, f_out)

def load_mnist_images(file_path):
    """Load MNIST images from the IDX file format."""
    with open(file_path, 'rb') as f:
        data = np.frombuffer(f.read(), np.uint8, offset=16)
    return data.reshape(-1, 784)

def load_mnist_labels(file_path):
    """Load MNIST labels from the IDX file format."""
    with open(file_path, 'rb') as f:
        data = np.frombuffer(f.read(), np.uint8, offset=8)
    return data

# Define paths
data_folder = '/tmp/mnist_data'
train_images_path = os.path.join(data_folder, 'train-images-idx3-ubyte')
train_labels_path = os.path.join(data_folder, 'train-labels-idx1-ubyte')
test_images_path = os.path.join(data_folder, 't10k-images-idx3-ubyte')
test_labels_path = os.path.join(data_folder, 't10k-labels-idx1-ubyte')

# Download and extract the MNIST dataset
if download_and_extract_mnist_zip(MNIST_ZIP_URL, data_folder):
    # Extract .gz files
    extract_gz_file(train_images_path + '.gz', train_images_path)
    extract_gz_file(train_labels_path + '.gz', train_labels_path)
    extract_gz_file(test_images_path + '.gz', test_images_path)
```

```
    extract_gz_file(test_labels_path + '.gz', test_labels_path)

    # Load data
    train_images = load_mnist_images(train_images_path)
    train_labels = load_mnist_labels(train_labels_path)
    test_images = load_mnist_images(test_images_path)
    test_labels = load_mnist_labels(test_labels_path)

    # One-hot encoding of labels
    def one_hot(labels, num_classes):
        return np.eye(num_classes)[labels]

    num_labels = 10  # MNIST has 10 classes
    train_labels = one_hot(train_labels, num_labels)
    test_labels = one_hot(test_labels, num_labels)

    print("MNIST dataset successfully downloaded and processed.")
else:
    print("Failed to download and process the MNIST dataset.")
```

```
MNIST dataset successfully downloaded and processed.
```

In [9]:

```
print('Train:', train_images.shape, train_labels.shape)
print('Test:', test_images.shape, test_labels.shape)
```

```
Train: (60000, 784) (60000, 10)
Test: (10000, 784) (10000, 10)
```

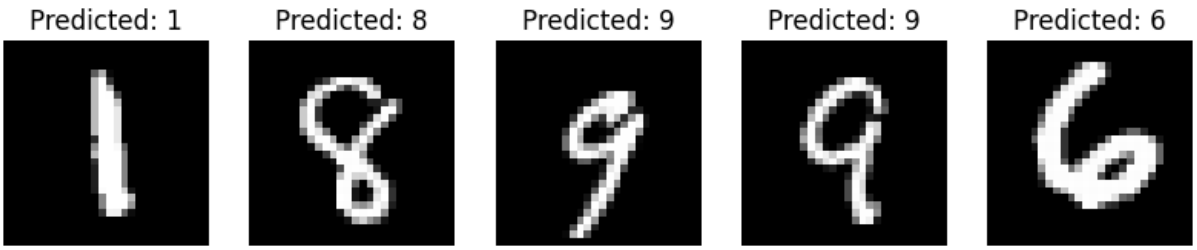
In [10]:

```
import time
import numpy as np

def get_train_batches(batch_size):
    """Yield batches of train data."""
    num_samples = train_images.shape[0]
    indices = np.arange(num_samples)
    np.random.shuffle(indices)  # shuffle the indices for randomness

    for start_idx in range(0, num_samples, batch_size):
        end_idx = min(start_idx + batch_size, num_samples)
        batch_indices = indices[start_idx:end_idx]
        yield train_images[batch_indices], train_labels[batch_indices]

# Assuming you have defined num_epochs, batch_size, and update functions
for epoch in range(num_epochs):
    start_time = time.time()
    for x, y in get_train_batches(batch_size):
        x = x.reshape(len(x), -1)  # Flatten the images if not already
        params = update(params, x, y)
    epoch_time = time.time() - start_time

    train_acc = accuracy(params, train_images.reshape(len(train_images), -1), train
    test_acc = accuracy(params, test_images.reshape(len(test_images), -1), test_lab
    print("Epoch {} in {:0.2f} sec".format(epoch, epoch_time))
    print("Training set accuracy {}".format(train_acc))
    print("Test set accuracy {}".format(test_acc))
```

```
Epoch 0 in 1.23 sec
Training set accuracy 0.9211333394050598
Test set accuracy 0.925599992275238
Epoch 1 in 0.98 sec
Training set accuracy 0.9387000203132629
Test set accuracy 0.9387999773025513
Epoch 2 in 0.95 sec
Training set accuracy 0.9493666887283325
Test set accuracy 0.9474999904632568
Epoch 3 in 0.97 sec
Training set accuracy 0.9588666558265686
Test set accuracy 0.9545999765396118
Epoch 4 in 1.01 sec
Training set accuracy 0.9646666646003723
Test set accuracy 0.9580000042915344
Epoch 5 in 0.98 sec
Training set accuracy 0.9681833386421204
Test set accuracy 0.9613999724388123
Epoch 6 in 1.00 sec
Training set accuracy 0.9719833731651306
Test set accuracy 0.9643999934196472
Epoch 7 in 1.00 sec
Training set accuracy 0.9759166836738586
Test set accuracy 0.9661999940872192
Epoch 8 in 0.99 sec
Training set accuracy 0.9773333668708801
Test set accuracy 0.9686999917030334
Epoch 9 in 1.00 sec
Training set accuracy 0.9798166751861572
Test set accuracy 0.9699999690055847
```

In [11]:

```python
import matplotlib.pyplot as plt
import numpy as np

# Placeholder for the model's predict function
# Replace this with your actual model's prediction method

# Select five random test images
num_samples = 5
indices = np.random.choice(len(test_images), num_samples, replace=False)
selected_images = test_images[indices]
selected_labels = test_labels[indices]

# Generate predictions
# Replace 'model' with your actual model
predicted_labels = np.argmax(batched_predict(params, selected_images),axis=1)

# Visualize the images and the predictions
plt.figure(figsize=(10, 2))
for i, (image, label) in enumerate(zip(selected_images, predicted_labels)):
    plt.subplot(1, num_samples, i + 1)
    plt.imshow(image.reshape(28, 28), cmap='gray')
    plt.title(f'Predicted: {label}')
    plt.axis('off')
plt.show()
```



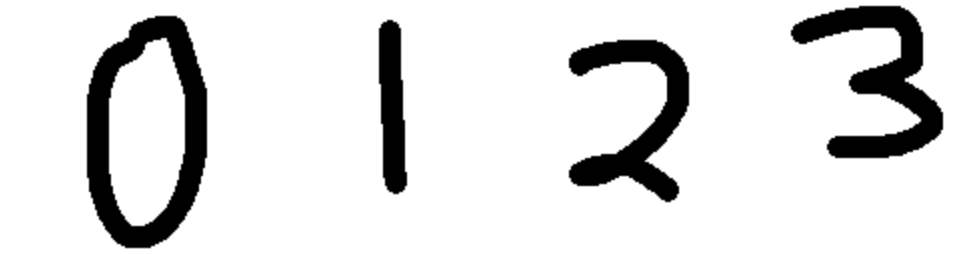
Next draw your own digits!

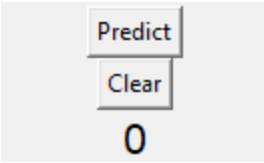
In [14]:
```
import tkinter as tk
from tkinter import Canvas
import numpy as np
from PIL import Image, ImageDraw


# Function to draw on the canvas
def draw(event):
    x, y = event.x, event.y
    canvas.create_oval((x, y, x + 10, y + 10), fill='black')
    draw_image.line([x, y, x + 1, y + 1], fill='black', width=10)

# Function to predict the digit
def predict_digit():
    canvas.postscript(file='digit.eps', colormode='color')
    img = Image.open('digit.eps')
    if img.mode != 'L':
        img = img.convert('L')
    img = img.resize((28, 28), Image.Resampling.LANCZOS)
    img = np.array(img)
    img = 1-img / 255.0

    digit = np.argmax(predict(params,np.ndarray.flatten(img)))
    label.config(text=str(digit))


# Function to clear the canvas and the image
def clear_canvas():
    canvas.delete("all")
    draw_image.rectangle((0, 0, 150, 150), fill='white')
    label.config(text="Draw a digit")

# Create the main window
root = tk.Tk()
root.title("Digit Predictor")

# Create a drawing canvas
canvas = Canvas(root, width=150, height=150, bg='white')
canvas.pack()

# PIL Image and Draw for capturing the drawn digit
image = Image.new("RGB", (150, 150), 'white')
draw_image = ImageDraw.Draw(image)
```

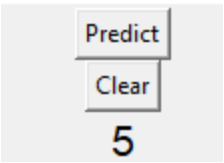
```
# Bind the drawing function to mouse drag
canvas.bind("<B1-Motion>", draw)

# Add a button to predict the digit
button_predict = tk.Button(root, text="Predict", command=predict_digit)
button_predict.pack()

# Add a button to clear the canvas
button_clear = tk.Button(root, text="Clear", command=clear_canvas)
button_clear.pack()

# Label to display the prediction
label = tk.Label(root, text="Draw a digit", font=("Helvetica", 16))
label.pack()

# Run the application
root.mainloop()
```

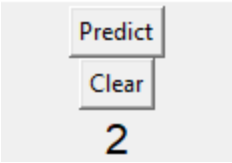
Discussion

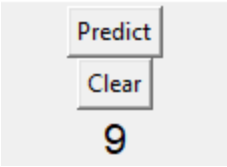
1. (1 point) How many layers are there in the neural network?
   There are 3 layers in the neural network: `[784, 512, 512, 10]`. `784` is the input dimension and hence isn't counted.
2. (1 point) What are the shapes of each parameter in "params"?
   Layer 1: `W1`: `(512, 784)`, `b1`: `(512,)`
   Layer 2: `W2`: `(512, 512)`, `b2`: `(512,)`
   Layer 3: `W3`: `(10, 512)`, `b3`: `(10,)`
3. (2 points) Which activation function is used?
   ReLU as defined in the `relu` function.
4. (2 points) What are the input space and label space?
   The input space is a flattened MNISTS image of 28x28 pixels, hence 784. In a batch it woudl be `(batch_size, 784)`. The output space is a one-hot vector of length 10 (each representing a digit 0-9).
5. (2 points) What does ``predict'' output?
   Returns a log-softmax output vector of length 10 each representing a digit 0-9, normalized so the probabilities sum to 1. The highest probability index indicates the predicted digit.
6. (2 points) What are the training accuracy and test accuracy after 10 epochs? Use the GUI, write your 1,2,3,4,...,10 using your mouse. Write a short report that shows your digits and the classification results.
   The training accuracy after 10 epochs is 0.98, the test accuracy is 0.97.

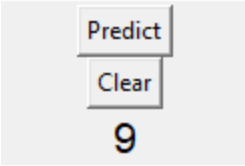


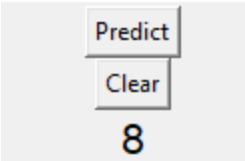


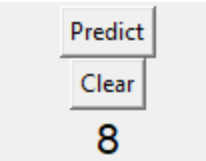


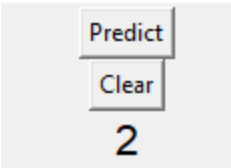


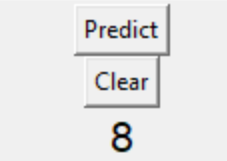


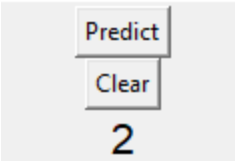












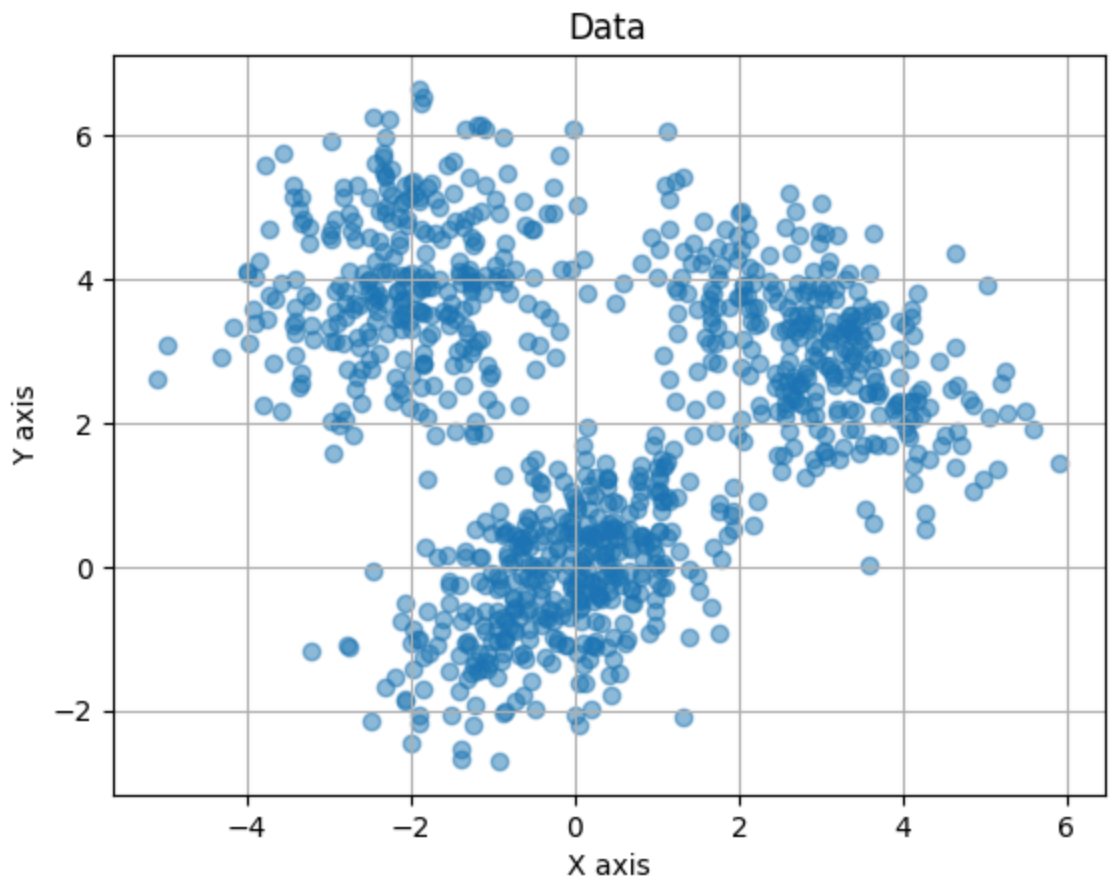
The model cleary struggles to correctly identify my handwritten digits. Do note that I changed some code for the PIL Resampling to `LANCZOS` instead of `ANTIALIAS` as it is

deprecated in my Pillow version. Online research says the algorithm is the same so it technically shouldn't affect results. It seems the model struggles to identify between the curves of each number and thus can't correctly associate each digit with its correct digit.

7. (2 points) What is the classification accuracy on your data? Is it better or worse than the accuracy on the test set? If it is very different, could you explain why? Any ideas to fix the issue?
   The classification accuracy is 30%. This is signifcantly worse than the accuracy on the test set. One would assume that with a test set accuracy of 97% the model would be able to identify my handwritten digits. We could fix this issue by providing more training data of varied digits. The best way to fix this would be to provide sets of training data of my own handwritten digits, and then train the model on this.

Q2. Implement the Lloyd's algorithm for k-means (30 points)

In [1]:
```
# First generate and visualize the input data


import numpy as np
import matplotlib.pyplot as plt

# Parameters for the mixture components
means = [np.array([0, 0]), np.array([3, 3]), np.array([-2, 4])]
covs = [np.array([[1, 0.5], [0.5, 1]]), np.array([[1, -0.5], [-0.5, 1]]), np.array(
weights = [0.4, 0.3, 0.3]  # Must sum to 1
num_samples = 1000

# Generate samples
samples = []
for mean, cov, weight in zip(means, covs, weights):
    num_component_samples = int(weight * num_samples)
    component_samples = np.random.multivariate_normal(mean, cov, num_component_samp
    samples.append(component_samples)

# Combine samples from all components
data = np.vstack(samples)

# Plot
plt.scatter(data[:, 0], data[:, 1], alpha=0.5)
plt.title('Data')
plt.xlabel('X axis')
plt.ylabel('Y axis')
plt.grid(True)
plt.show()
```



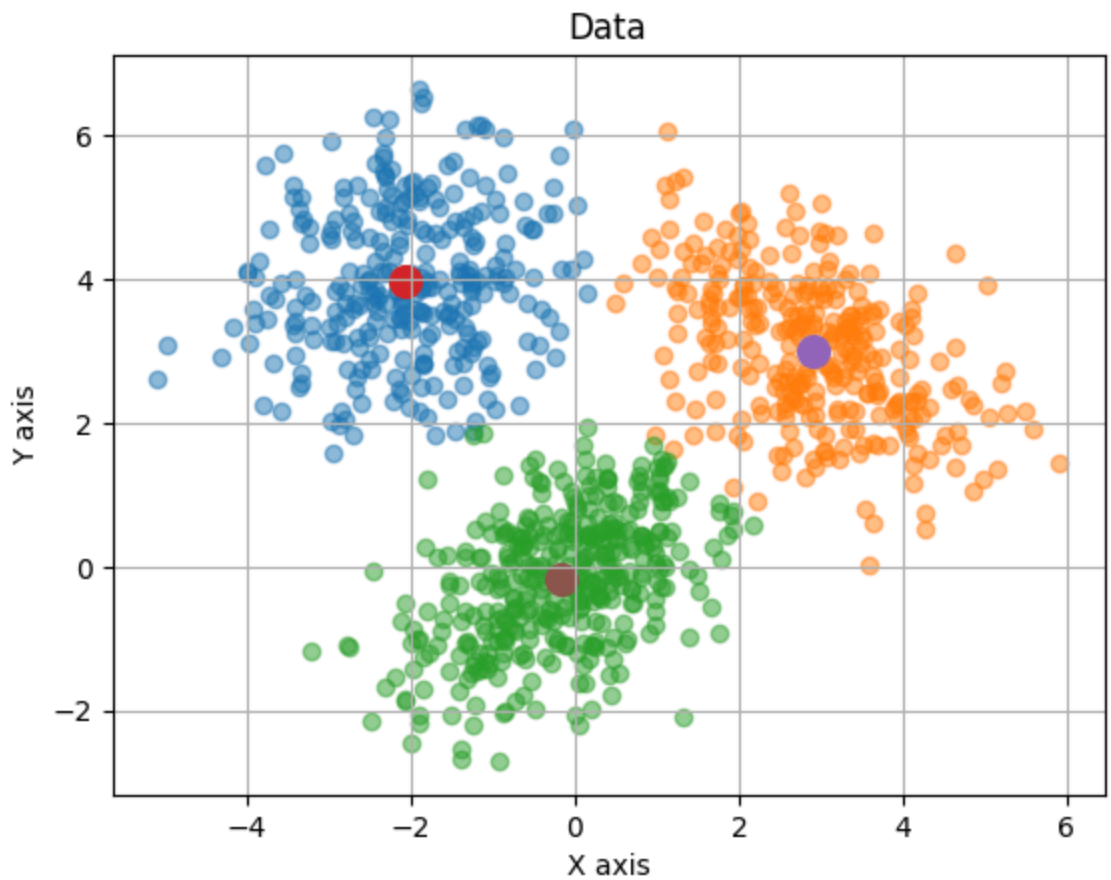
In [2]:
```
import numpy as np
import matplotlib.pyplot as plt

def initialize_centroids(data, k):
    # Randomly select k data points from the dataset as centroids
    # ==================YOUR CODE HERE =================


    indices = np.random.choice(data.shape[0], k, replace=False)


    return data[indices]

def assign_clusters(data, centroids):
    # Return a vector of integers in {0,1,2,3,...,k-1} to indicate the cluster assi
    # the vector should be of length as long as the data size

    # ==================YOUR CODE HERE =================

    clusters = np.zeros(data.shape[0], dtype=int)
    for i in range(data.shape[0]):
        distances = np.linalg.norm(data[i] - centroids, axis=1)
        clusters[i] = np.argmin(distances)

    # ==================================================
    return clusters

def update_centroids(data, clusters, k):
```

```
    # update the centroids
    # ==================YOUR CODE HERE =================

    new_centroids = np.zeros((k, data.shape[1]))
    for i in range(k):
        cluster_points = data[clusters == i]
        if len(cluster_points) > 0:
            new_centroids[i] = np.mean(cluster_points, axis=0)
        else:
            new_centroids[i] = data[np.random.choice(data.shape[0])]

    # ==================================================
    return new_centroids

def kmeans(data, k, max_iters=100):
    centroids = initialize_centroids(data, k)
    # Write a for loop to implement the kmeans algorithm
    for _ in range(max_iters):
        # ==================YOUR CODE HERE =================

        clusters = assign_clusters(data, centroids)
        centroids = update_centroids(data, clusters, k)

        # ==================================================
    # return the updated cluster centers
    return centroids, clusters
```

In [11]:

```
# If your implementation is valid, then the following should generate the clusterin

k=3
centroids, clusters = kmeans(data,k)
print("Centroids:", centroids)
# Plot
colors =  plt.get_cmap("tab10")

for i in range(k):
    plt.scatter(data[clusters==i, 0], data[clusters==i, 1], color=colors(i), alpha=
for i in range(k):
    plt.scatter(centroids[i,0],centroids[i,1],marker='o',s=128,color=colors(i+k))
plt.title('Data')
plt.xlabel('X axis')
plt.ylabel('Y axis')
plt.grid(True)
plt.show()
```

```
Centroids: [[-2.06420474  3.97617221]
 [ 2.89973113  3.01793935]
 [-0.17789798 -0.17712978]]
```



Next, let's do clustering of a real data set "Olivetti faces". The same function you implemented above should work on this dataset directly without changes.

1. Do you get the same results each time you run k-means? Why?
   No, we do not get the same results each time we run k-means. This is because k-means is sensitive to the random initialization of the centroids. If we were to set the random seed, then we should obtain consistent results across runs. A random init each run results in a potentially different local minima in the objective function, resulting in different clusters.

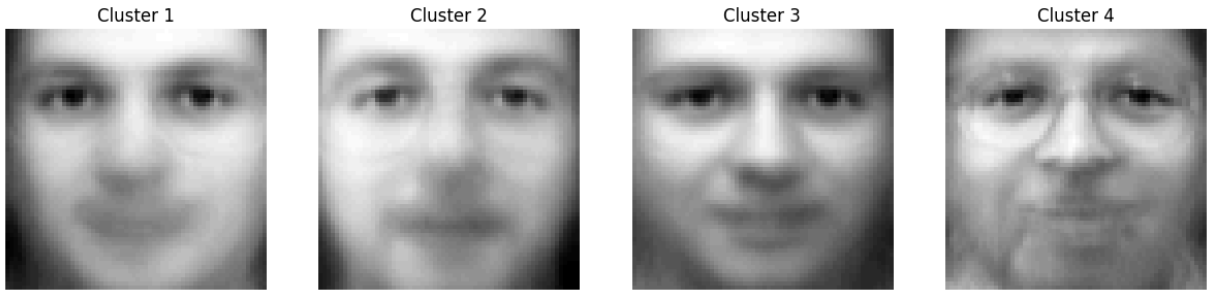
2. Describe what you see when you adjust k. You may reorganize the subplot layouts to see clearly.
   For small `k` (< 10), the centroids are quite blurry as each centroid averages across many different faces. These centroids only capture very broad details such as lighting and general face shape. As we increase `k` the centroids become more recognizable and start capturing more facial features such as glasses and face direction. Once `k` becomes extremely large (25+), each centroid almost represents an entirely unique person. The centroids are much more structured and meaningful.

3. What are the prominent differences you can see between the raw images and the "centroid faces"?
   Raw images are real individual faces, with a clear identity and fine detail. The centroid faces are blurry, with more smoothed features (as they are averaged). The faces also look more symmetric and even. There are clear differences in lighting and key features (glasses, hair, wrinkles, facial shape).

In [15]:
```
from sklearn.datasets import fetch_olivetti_faces

# We'll fetch the Olivetti faces dataset
data = fetch_olivetti_faces().data

k = 4  # number of clusters

random_imgs = initialize_centroids(data, k)

# Visualize the cluster centers as images
plt.figure(figsize=(15, 5))
for i, centroid in enumerate(random_imgs ):
    plt.subplot(1, k, i + 1)
    plt.imshow(centroid.reshape(64, 64), cmap='gray')
    plt.title(f'Cluster {i+1}')
    plt.axis('off')
plt.suptitle('random raw images')
plt.show()




# Apply K-Means Clustering
centroids, _ = kmeans(data, k)

# Visualize the cluster centers as images
plt.figure(figsize=(15, 5))
for i, centroid in enumerate(centroids):
    plt.subplot(1, k, i + 1)
    plt.imshow(centroid.reshape(64, 64), cmap='gray')
    plt.title(f'Cluster {i+1}')
    plt.axis('off')
plt.suptitle('K-Means Cluster Centers')
plt.show()
```

random raw images

K-Means Cluster Centers



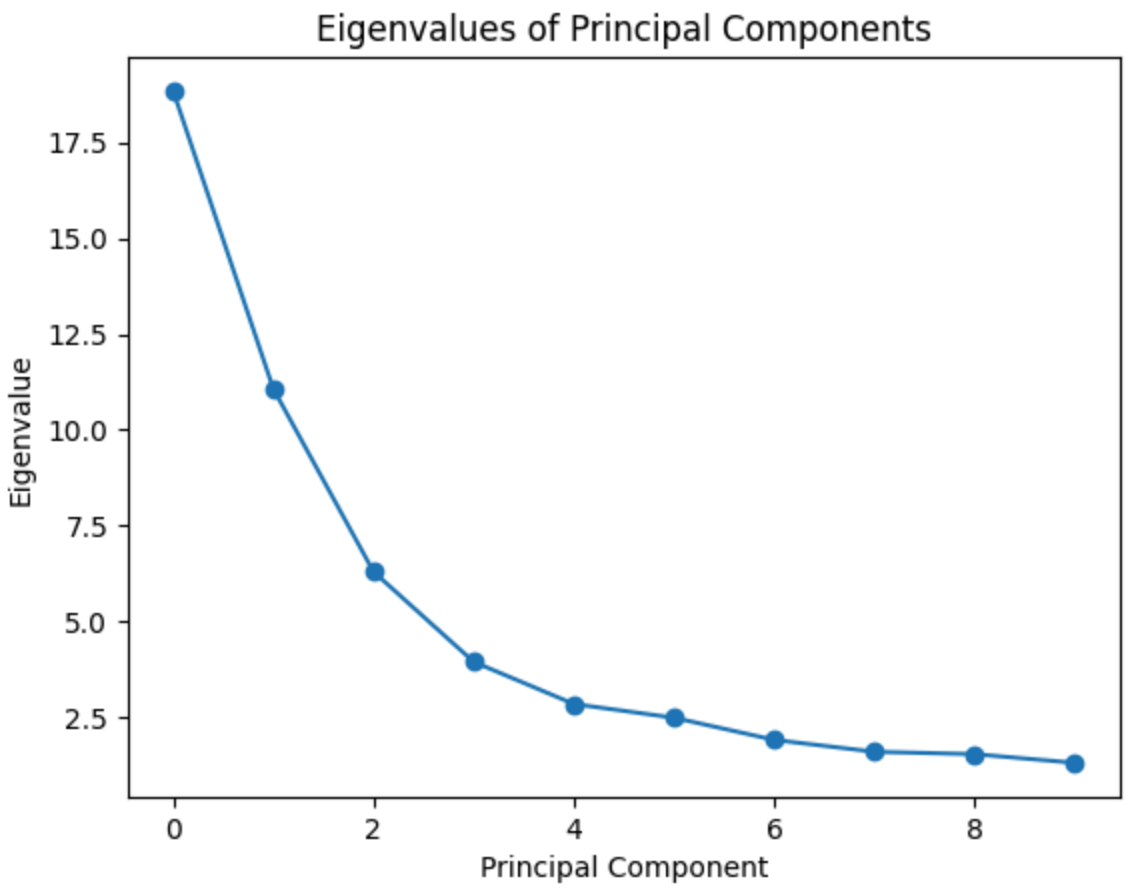
Q3. PCA for Dimension Reduction (30 points)

In this problem, we will implement the Principal Component analysis.

In [3]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_olivetti_faces


def pca(data, num_components):

    # ==================YOUR CODE HERE =================

    # Remember PCA involves
    # (1)estimate and substract the mean,
    # (2) construct the covariance matarix
    # (3) Find the eigenvectors corresponding to the top k eigenvalues
    # Return the eigenvalues and eigenvectors
    # ==================================================

    # Step 1: Compute the mean of the data
    mean_vector = np.mean(data, axis=0, keepdims=True)
    # Step 2: Center the data by subtracting the mean
    centered_data = data - mean_vector
    # Step 3: Compute the covariance matrix
    covariance_matrix = np.cov(centered_data, rowvar=False)
    # Step 4: Compute the eigenvalues and eigenvectors
    eigenvalues, eigenvectors = np.linalg.eigh(covariance_matrix)
    # Step 5: Sort the eigenvalues and eigenvectors in descending order
    sorted_indices = np.argsort(eigenvalues)[::-1]
    sorted_eigenvalues = eigenvalues[sorted_indices]
    sorted_eigenvectors = eigenvectors[:, sorted_indices]
    # Step 6: Select the top k eigenvalues and corresponding eigenvectors
    top_eigenvalues = sorted_eigenvalues[:num_components]
    top_eigenvectors = sorted_eigenvectors[:, :num_components]

    return top_eigenvalues, top_eigenvectors
```

```
# The following code help you to visualize your eigenvectors

data = fetch_olivetti_faces().data

num_components = 10  # Number of principal components
eigenvalues, principal_components = pca(data, num_components)
```

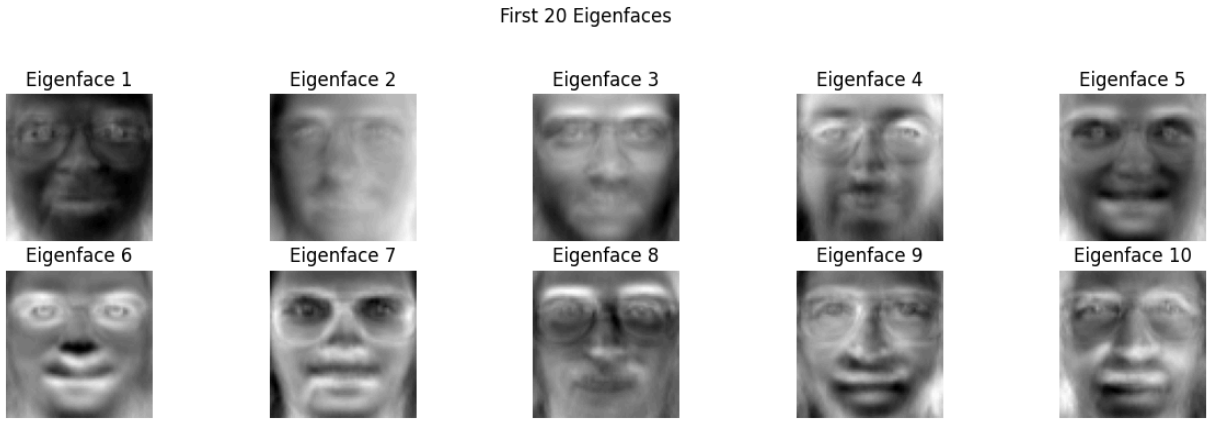
In [4]:
```
# ==================YOUR CODE HERE =================

# Plot the eigenvalues in descending orders.

# ==================================================
plt.plot(eigenvalues, marker='o')
plt.xlabel('Principal Component')
plt.ylabel('Eigenvalue')
plt.title('Eigenvalues of Principal Components')
plt.show()
```



In [5]:
```
# Eigenfaces would be the principal components reshaped as images
eigenfaces = principal_components.T.reshape((-1, 64, 64))
print("Eigenfaces shape:", eigenfaces.shape)

# Visualize the first 20 eigenfaces
plt.figure(figsize=(15, 8))
for i, eigenface in enumerate(eigenfaces):
    plt.subplot(4, 5, i + 1)
    plt.imshow(eigenface.reshape(64, 64), cmap='gray')
```

```
    plt.title(f'Eigenface {i+1}')
    plt.axis('off')
plt.suptitle('First 20 Eigenfaces')
plt.show()
```

```
Eigenfaces shape: (10, 64, 64)
```



Answer the following questions

1. When you rerun PCA with different choices of "number_of_components" do they return the same or different solutions? Why?
   The principal components themselves stay the same, but the number of components we keep obviously changes. PCA computes the eigenvectors of the covariance matrix, which will always be uniquely determined. Requesting a different number of components doesn't return different eigenvectors, as we just truncate the number of results to return. Numerical precision and sign may change a little, however this overall doesn't change the subspace.

2. How would you interpret the eigenfaces above?
   We can interpret each eigenface as the principal directions of variance in the "face space." This means tha the first few eigenfaces capture lighting direction, overall face structure, pose, while higher components capture finer details such as facial features, wrinkles, glasses, etc.